

INFORME DE EJECUCIÓN — FASE 1 “SEAWOLF NEREUS”

Cirugía del rehén: sustitución del servicio cerrado por persistencia propia

Fecha: 2026-09-30 → 2026-10-01 · **Prioridad:** MÁXIMA · **Clasificación:** CRÍTICO —
Uso Interno Seawolf Studio **Elaborado por:** Bellion (Gran Comandante) — misión en
solitario · **Encargado por:** Monarca (Key) **Objetivo:** arrancar la dependencia de
CopilotKit Intelligence (servicio cerrado, obligatorio, fuera de la licencia MIT) y
sustituirla por **persistencia propia (PostgreSQL + pgvector)** para hilos y replay.
Resultado: ☒ **FASE 1 COMPLETADA.** El servidor arranca **SIN la clave cerrada**, los
hilos y el replay viven en **Postgres propio**, y la suite sigue en **276/276 verdes**.

1. RECONOCIMIENTO — DÓNDE ESTABA EL REHÉN

El aferramiento estaba **concentrado en 4 puntos**: | Archivo | Acoplamiento | |———|
———| | apps/server/src/config.ts | required("CPK_INTELLIGENCE_API_KEY") en
readConfig + assertApiDeploymentConfig **abortaba el arranque** si faltaba la clave | |
apps/server/src/app.ts | new CopilotKitIntelligence({...}),
intelligence.getOrCreateThread(...) en /api/main-thread | |
apps/server/src/agent.ts | new CopilotRuntime({ agents, intelligence, ... })
(modo Intelligence) | | apps/server/src/demo/entry.ts | exigía la clave para el demo |

Hallazgo decisivo: el runtime v2 de CopilotKit ofrece **dos modos** — *Intelligence* (hilos
durables vía el servicio hospedado) y **SSE con un AgentRunner propio** (intelligence?:
undefined). El paquete expone además la interfaz **LocalThreadEndpointRunner** (con
listThreads/getThreadMessages/getThreadEvents/getThreadState/clearThreads)
documentada explícitamente para “*usar el mismo sobre HTTP que el backend de
Intelligence*”. Esa es la puerta de salida.

2. LO QUE SE CONSTRUYÓ

2.1 Infraestructura

- **Postgres 17 + pgvector** en Docker, **aislado**: contenedor nereus-postgres, puerto
127.0.0.1:5433 (solo loopback), volumen nereus_pgdata, --restart unless-
stopped. **No se tocó ningún servicio previo del VPS.**

- Extensión **pgvector 0.8.6** habilitada (lista para la memoria de la Fase 4).

2.2 Código nuevo — apps/server/src/runner/store-runner.ts (220 líneas)

StoreAgentRunner implementa AgentRunner + LocalThreadEndpointRunner: - **Composición** sobre InMemoryAgentRunner para la mecánica de run/stream (no reinventa concurrencia ni leases). - **Persistencia write-through** al Store propio del proyecto (misma capa records que ya usan tareas/aprobaciones) — sin tablas nuevas, reutilizando código probado. - **Eventos**: se observan mientras se transmiten y se **anexan una sola vez** por run (sin duplicar en replay). - **Mensajes**: se reemplazan con el snapshot completo de la conversación al cerrar cada run. - **Hidratación** al arranque (init()): reconstruye el índice de hilos desde Postgres. - Endpoints de hilos (listThreads/getThreadMessages/getThreadEvents/getThreadState/clearThreads) + ensureThread/updateThread (renombrar/archivar).

2.3 Cableado — estrategia dual-mode

Modo	Cuándo	Efecto
NEREUS (propio)	CPK_INTELLIGENCE_API_KEY ausente	runner: StoreAgentRunner (SSE) — sin servicio cerrado
Legacy (paridad)	clave presente	intelligence hospedado (comportamiento original intacto)

Esto preserva el 100% de la suite existente y deja el servicio cerrado como **opcional**, nunca requerido. **NEREUS no lo usa.**

- config.ts: la clave pasó a process.env.CPK_INTELLIGENCE_API_KEY?.trim() (opcional); assertApiDeploymentConfig ya no aborta sin ella.
- app.ts: construye el runner cuando no hay clave; /api/main-thread asegura el hilo en **nuestro** store.
- agent.ts: ThreadBackend = { intelligence } | { runner }, runtime bifurcado.
- demo/entry.ts: ya no exige la clave.

3. VERIFICACIÓN REAL (evidencia en disco y contra la base)

Prueba	Resultado
pnpm typecheck (raíz + worker)	✅ EXIT=0
pnpm lint (Biome, 133 archivos)	✅ limpio
pnpm build:server	✅ EXIT=0
pnpm test (suite completa)	✅ 276/276 pasan · 0 fallos

Prueba	Resultado
Arranque sin CPK_INTELLIGENCE_API_KEY /api/main-thread (run 1)	✓ servidor vivo, /api/health → {"ok":true,"mode":"sample"} ✓ threadId=663f0172-...
Reinicio del proceso → /api/main-thread (run 2)	✓ mismo threadId (persistido en Postgres)
Fila en Postgres	✓ kind=nereus-threads, owner=local-user
/api/copilotkit/threads (GET)	✓ devuelve el hilo desde nuestro store (endpoints locales activos)
Run AG-UI real (/api/copilotkit/agent/default/run)	✓ stream RUN_STARTED→...→RUN_FINISHED
Persistencia del run	✓ fila tras el run: 9 eventos + 3 mensajes
Llamadas al servicio cerrado en logs	✓ 0 (cero AUTH_UNAUTHENTICATED)

Conclusión: el rehén está arrancado. El agente corre con persistencia propia; los hilos y el replay sobreviven al reinicio sin ningún servicio externo.

4. ARCHIVOS MODIFICADOS / CREADOS

Archivo	Cambio
apps/server/src/runner/store-runner.ts	NUEVO — runner con persistencia propia
apps/server/src/config.ts	clave cerrada → opcional; guard de despliegue relajado
apps/server/src/app.ts	runner + /api/main-thread sobre store propio; dual-mode
apps/server/src/agent.ts	ThreadBackend; runtime SSE o Intelligence
apps/server/src/demo/entry.ts	sin exigencia de la clave
tests/config.test.ts	tests actualizados a la nueva arquitectura

5. LÍMITES HONESTOS (lo que NO está resuelto aún)

1. **Granularidad de almacenamiento:** cada hilo se guarda en **una sola fila jsonb** (eventos+mensajes). Correcto y suficiente ahora; crecerá con el uso → mejora 1.x: filas por run + poda/paginación.
2. **Sesiones SSE en un solo proceso:** la caché de hilos es por proceso; el multi-instancia no está probado (no era objetivo aún).
3. **Modelo real:** el run probado usó el **agente sample** (sin modelo). El pipeline de modelo entra en la **Fase 2 (OpenRouter)**.

4. **Vía legacy** de Intelligence conservada por paridad; NEREUS no la usa.
 5. **DATABASE_URL** guardada en `.env` (`chmod 600`) del VPS; el servicio aún no está bajo `systemd` (se levanta a mano en las pruebas).
-

6. ESTADO Y PENDIENTES

-  **Fase 0** — clone + build (276 tests, binario ejecutable).
 -  **Fase 1** — **persistencia propia; servicio cerrado arrancado.**
 -  **Fase 2** — Gateway de modelo **OpenRouter** (adiós al coste por token obligatorio).
 -  Endurecer operación (`systemd` + rotación de credenciales).
 -  Reporte para auditoría de **Beru**.
-

7. VEREDICTO DEL COMANDANTE

El muro que bloqueaba el arranque total del proyecto **ha caído**: NEREUS ya tiene su propia memoria de hilos, sin rehén externo. La suite completa sigue verde tras la cirugía — señal de que se operó sin daño colateral. **La cancha está lista para el motor de modelo (Fase 2).**

| **Recomendación:** autorizar **Fase 2** — **OpenRouter** como gateway único.

Fin del informe de Fase 1. Listo para auditoría de Beru.